MLOps Pipeline for Market Regime Detection

MLOps Final Project Report

RAJAVIVEGAN M R - NA22B085

April 28, 2026

Abstract

This report documents the end-to-end design, development, and operational deployment of a market regime detection platform. The core idea started as a research challenge: can statistical regime-detection methods that succeed at high-frequency (sub-second) resolution be adapted to daily OHLCV data sourced freely? We found that classical unsupervised methods such as Hidden Markov Models degrade sharply at daily granularity, where macroeconomic inertia dominates short-run microstructure. The project therefore pivots to a supervised, feature-engineered XGBoost pipeline trained on carefully constructed daily indicators. The resulting system classifies three orthogonal regime dimensions — trend versus mean-reversion, high versus low volatility, and bull versus bear market direction — and serves these predictions through a fully containerised MLOps stack comprising FastAPI, MLflow, DVC, Prometheus, Grafana, and Airflow. This document presents all five required deliverables alongside a detailed narrative of experimental decisions, challenges faced, and how each MLOps tool is wired into the operational loop.

Contents

1 Introduction and Problem Motivation **3**
- 1.1 The Original Research Question . . . 3
- 1.2 The Pivot to Daily Equity Data . . . 3
- 1.3 Why Standard Models Outperformed Unsupervised Methods Here . . . 3

2 Experimental Journey: From Notebooks to Pipeline **3**
- 2.1 Phase 1 — Exploratory Analysis in Colab . . . 3
  - 2.1.1 Label Generation Strategy . . . 4
  - 2.1.2 Data Leakage Discovery and Resolution . . . 5
- 2.2 Phase 2 — Modular Production Code . . . 5

3 System Architecture **6**
- 3.1 High-Level Architecture Diagram . . . 6
- 3.2 Component Table . . . 6
- 3.3 Data Flow Narrative . . . 6

4 Deliverable 1: Architecture Diagram with Block Explanations **7**

5 Deliverable 2: High-Level Design **8**
- 5.1 Design Philosophy . . . 8
- 5.2 Technology Choice Rationale . . . 8
- 5.3 ML Metric and Business Metric Targets . . . 9

6 Deliverable 3: Low-Level Design **9**
6.1 API Endpoint Specification . . . 9
6.2 Request / Response Schemas . . . 9
6.3 Internal Module Dependency Graph . . . 10
6.4 Error Codes . . . 10

7 MLOps Tooling: Technical Wiring **10**
7.1 DVC Pipeline and Reproducibility . . . 10
7.2 MLflow Experiment Tracking . . . 11
7.3 Prometheus Instrumentation . . . 11
7.4 Grafana Dashboard . . . 12
7.5 Retraining and Alert Logic — End-to-End Technical Detail . . . 12
7.5.1 Drift Detection . . . 12
7.5.2 Retraining Gating . . . 12
7.5.3 Retraining Execution . . . 12
7.5.4 Prometheus and Grafana Alert Path . . . 13

8 Live Simulation: Mimicking Production **13**
8.1 What the Simulation Represents . . . 13
8.2 How This Maps to Real Production Use . . . 13
8.3 Extensions for True Production . . . 14

9 Model Results and Metric Justification **14**
9.1 Final Reported Scores . . . 14
9.2 Why These Numbers Are Correct . . . 14

10 The User Interface **15**

11 Challenges Faced and How They Were Resolved **16**

12 Deliverable 4: Test Plan and Test Cases **17**
12.1 Test Objectives . . . 17
12.2 Test Cases . . . 17
12.3 Execution . . . 17
12.4 Acceptance Criteria . . . 18

13 Deliverable 5: User Manual **18**
13.1 What This Application Does . . . 18
13.2 Starting the Application . . . 18
13.3 Using the Prediction Screen . . . 18
13.4 Using the Pipeline Screen . . . 18
13.5 Troubleshooting . . . 18

14 Future Extensions **19**

15 AI Disclosure Agreement **19**

1 Introduction and Problem Motivation

1.1 The Original Research Question

The initial motivation for this work was a research-level problem: *can market regime be detected in real time from second-level tick data?* In that setting, a regime detector could theoretically adjust algorithmic order routing mid-session, switching from momentum to market-making strategies as the microstructure transitions. The scientific literature on this topic uses HMMs, regime-switching GARCH, and UMAP-based clustering applied to order-flow imbalance and intraday volatility signatures.

The practical constraint that forced a pivot was data availability. Second-level trade data with sufficient history is commercially licensed. The only freely available second-level data at scale comes from major cryptocurrency exchanges, primarily through the Binance API. However, as our own experiments confirmed, the structural properties of cryptocurrency markets make classical regime detection unreliable there: the underlying instrument is fundamentally trend-dominated with no macroeconomic anchoring, so HMMs trained on crypto return sequences tend to detect volatility clusters rather than economically meaningful regimes.

1.2 The Pivot to Daily Equity Data

Daily OHLCV data for equities is freely accessible through `yfinance`. Crucially, at the daily time-scale, the statistical properties that make regime detection tractable are better preserved. Macroeconomic cycles play out over weeks and months rather than microseconds. Volatility regimes are persistent and autocorrelated. Trend and mean-reversion can be identified using well-studied measures such as the Hurst exponent and variance ratio. And the output of a next-day regime classifier has genuine practical utility: a portfolio manager who knows whether tomorrow is likely to be a trending or a mean-reverting session can adjust their strategy selection accordingly.

The project therefore became: *can we build a robust, fully operational MLOps pipeline that classifies the next-day market regime for an equity index using freely available daily data?*

1.3 Why Standard Models Outperformed Unsupervised Methods Here

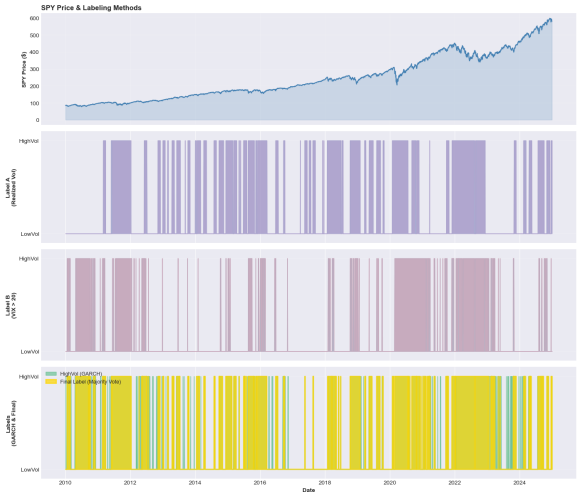
A significant part of the experimental work involved testing whether HMMs could serve as the primary regime classifier. The result was consistent: HMMs trained on daily SPY returns achieved approximately 60–70% directional accuracy on held-out data but were sensitive to initialization, produced unstable state sequences, and were difficult to integrate into a reproducible MLOps pipeline because model artifacts cannot be serialized in a standard, registry-friendly format. More importantly, the Hurst exponent and variance ratio, which are the most reliable unsupervised regime indicators at daily frequency, turned out to be better used as label-generation tools rather than as models themselves. Once those methods defined clean binary training labels, a supervised XGBoost classifier with balanced sample weighting consistently outperformed the HMM baseline. The final design therefore uses unsupervised statistics for labeling and supervised learning for prediction.

2 Experimental Journey: From Notebooks to Pipeline

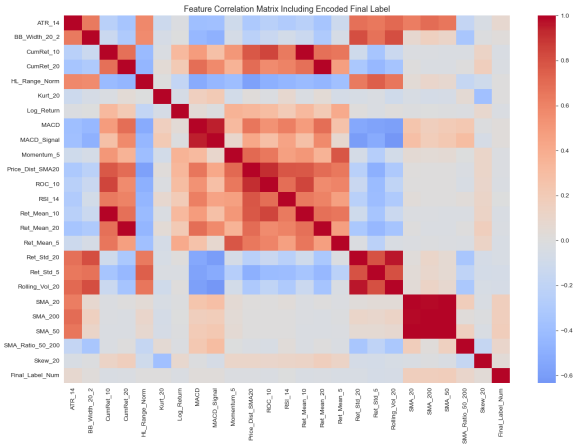
2.1 Phase 1 — Exploratory Analysis in Colab

Three Jupyter notebooks document the experimental phase: `trend_regime_eda.ipynb`, `volatility_regime_de` and `bull_vs_bear.ipynb`. Each notebook explored a different regime dimension independently,

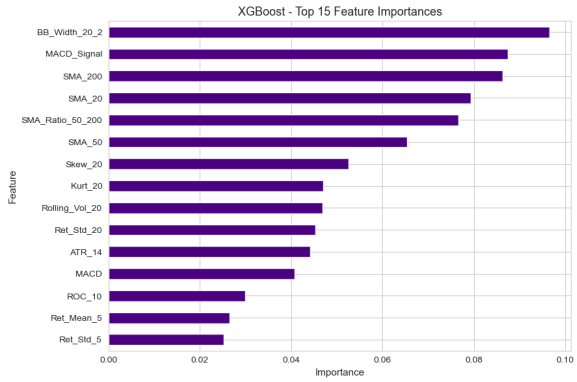
following the same structure: download data, define labels via multiple methods, build features, compare models, and evaluate on a held-out time window.



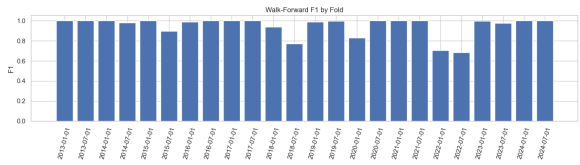
(a) Regime label overlay on SPY price



(b) Feature correlation heatmap



(c) XGBoost feature importance (trend notebook)



(d) Walkforward validation F1 scores

Figure 1: Exploratory Data Analysis Grid from the three regime notebooks.

2.1.1 Label Generation Strategy

For the trend regime, three independent labeling approaches were combined via majority vote:

- **Hurst exponent** (R/S analysis, 60-day rolling window): $H > 0.55 \Rightarrow$ Trending; $H < 0.45 \Rightarrow$ MeanReverting.
- **Variance ratio test** (60-day window, $k = 5$): ratio $> 1.1 \Rightarrow$ Trending; ratio $< 0.9 \Rightarrow$ MeanReverting.
- **Lag-1 autocorrelation** (30-day rolling): autocorr $> 0.1 \Rightarrow$ Trending; autocorr $< -0.1 \Rightarrow$ MeanReverting.

Days where all three methods disagreed were labelled "Neutral" and *excluded from training*. This is a principled choice: the supervised model is only trained on days where the regime signal is unambiguous, so it learns from the cleaner portion of the distribution.

For volatility, majority vote combined a realized-volatility threshold (20-day rolling std versus its 252-day median), a VIX threshold (> 20 for high volatility), and a GARCH(1,1) conditional volatility threshold. The GARCH model fitted on the full training window produced the following estimates for SPY:

$$\sigma_t^2 = \underbrace{0.0377}_{\omega} + \underbrace{0.1692}_{\alpha}\,\varepsilon_{t-1}^2 + \underbrace{0.7987}_{\beta}\,\sigma_{t-1}^2 \tag{1}$$

The high β coefficient (0.799) confirms strong volatility clustering, validating that regime persistence is a real feature of the data rather than a modelling artefact. The sum $\alpha + \beta = 0.968$ indicates near-integrated GARCH behaviour, which is consistent with the well-known volatility persistence of equity indices.

For the bull/bear regime, the primary label comes from the peak-to-trough drawdown rule: a day is labelled *Bear* if SPY is more than 20% below its rolling 252-day high. This is the standard definition used by major financial institutions and avoids the data-leakage problem of using the 200-day SMA as both a feature and a label.

2.1.2 Data Leakage Discovery and Resolution

The most important finding from the notebook phase was a labelling leakage problem. In the initial bull/bear notebook, the label was derived from the 200-day SMA crossing rule, and features included the SMA-50/SMA-200 ratio. This created a direct mathematical path from label to feature, explaining why early models showed near-perfect accuracy ($F_1 > 0.97$). Recognising this, the label was changed to the drawdown rule and all SMA-200-based features were removed. The resulting test F_1 dropped to 0.69, which is the honest, leak-free score.

2.2 Phase 2 — Modular Production Code

After the notebook experiments, all logic was refactored into five independent modules under `src/`:

- `data_ingestion.py` — download, validate, save raw CSVs, compute baseline statistics.
- `feature_engineering.py` — centralized feature builder used by both training and inference.
- `train.py` — time-split, scale, train, evaluate, register.
- `predict.py` — live inference entry point used by the API.
- `drift_monitor.py` — KL-divergence-based drift detection against stored baselines.

The most important design rule is that `feature_engineering.py` is shared. Training and inference use the same code to build the same features from the same rules, which eliminates train/serve feature skew by construction.

3 System Architecture

3.1 High-Level Architecture Diagram

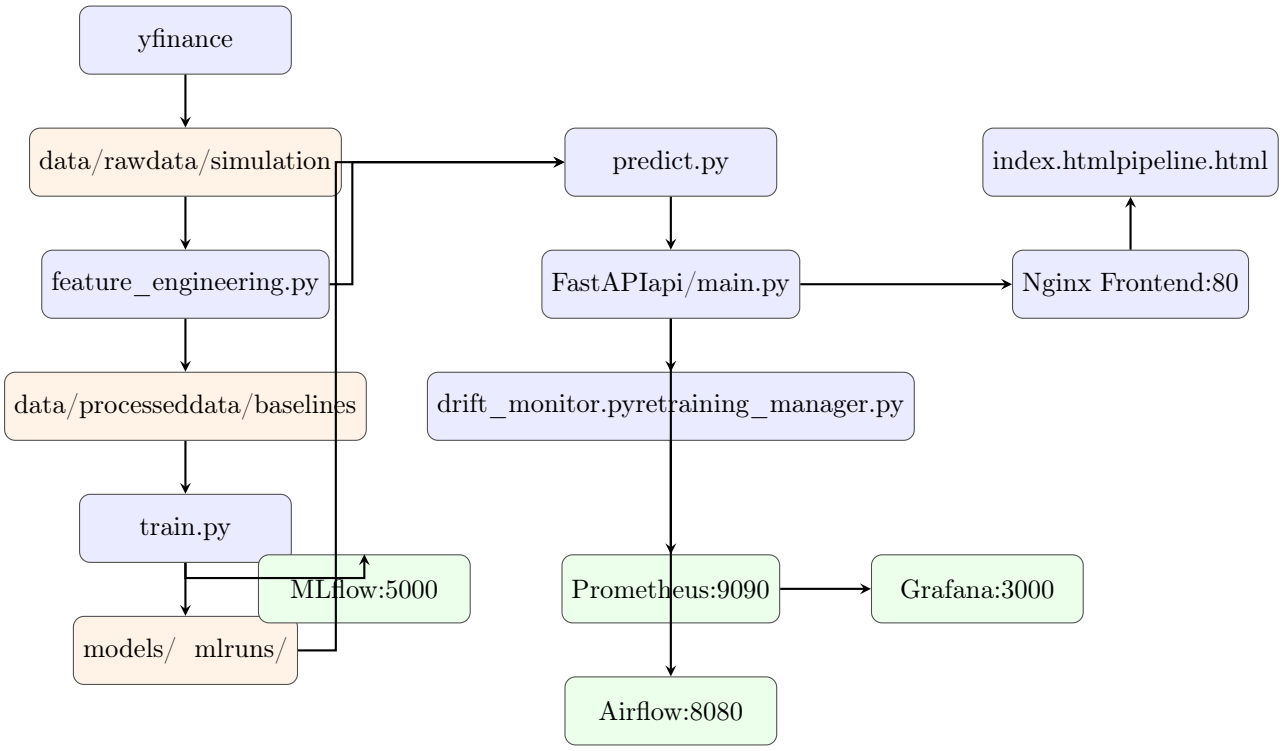


Figure 2: Full system architecture. Blue boxes are compute components; orange are data stores; green are observability services.

3.2 Component Table

Component	Port	Responsibility
Nginx Frontend	80	Serves index.html and pipeline.html; routes /pipeline to pipeline.html
FastAPI Backend	8000	REST API: health, prediction, drift, retrain, history
Prometheus Exporter	8001	Scraped by Prometheus; exposes all custom metrics
MLflow Server	5000	Experiment tracker and model registry
Prometheus	9090	Time-series metrics store; evaluates alert rules
Grafana	3000	Dashboarding and near-real-time visualization
Airflow	8080	DAG-based pipeline orchestration

Table 1: Docker Compose service topology.

3.3 Data Flow Narrative

The operational sequence proceeds as follows. Raw market data is downloaded from Yahoo Finance and written to `data/raw/`. `feature_engineering.py` reads those CSVs, applies rolling statistics, oscillators, volatility estimators, and cross-asset signals, and writes the result to `data/processed/`. During that step it also records the mean and standard deviation of each feature into `data/baselines/feature_baselines.json`, which is the reference distribution for drift detection.

`train.py` then reads the processed tables, performs a strict time-ordered split, scales features

using a `StandardScaler` fitted only on the training partition, trains XGBoost classifiers with balanced sample weights, and writes serialised artifacts to `models/`. Every training run is logged to the MLflow server with parameters, metrics, confusion-matrix plots, and feature importance charts.

When the API receives a prediction request, `predict.py` downloads the most recent 300 days of market data from yfinance, passes them through the same feature-engineering functions used in training, extracts the most recent complete feature row, scales it with the saved scaler, and asks the loaded model for a class probability vector. The predicted label and confidence score are returned to the frontend.

Prometheus scrapes the metrics endpoint on port 8001 every 15 seconds. Grafana queries Prometheus and renders the operational dashboard. When drift is detected through the `/drift/{regime}` endpoint, the drift score gauge updates in Prometheus, Grafana evaluates the alert rule, and if the threshold is exceeded the retraining manager is triggered.

4 Deliverable 1: Architecture Diagram with Block Explanations

Figure 2 is the primary architecture diagram. The following explains each block.

yfinance / Data Ingestion. `src/data_ingestion.py` wraps the `yfinance` library with retry logic (up to three attempts with exponential back-off), freshness checking (skips re-download if the file is less than 24 hours old), and a per-ticker validation report. It downloads training data (2010–2022) for SPY, QQQ, IWM, GLD, TLT, and VIX, and separately downloads the held-out simulation window (2023–2024) for SPY. It also computes baseline statistics that will later be used for drift detection.

Feature Engineering. `src/feature_engineering.py` is the single source of truth for all feature computation. It is intentionally shared between training and live inference so that both sides compute features identically. The module implements RSI (Wilder smoothing, manual), ATR, MACD, Bollinger Bands, Parkinson and Garman-Klass volatility estimators, rolling autocorrelation, rolling skewness/kurtosis, and cross-asset signals (TLT/SPY bond-equity correlation, IWM/SPY breadth ratio). All entry-point functions call an internal `_ensure_flat_ohlcv()` normaliser that collapses any yfinance MultiIndex column headers into standard flat names before any computation runs.

Training and Registry. `src/train.py` performs a time-ordered train/val/test split, fits a `StandardScaler` on the training partition only, trains XGBoost with balanced sample weights to handle class imbalance, evaluates on the validation and test partitions, and logs everything to MLflow. Class labels are stored in a `.json` sidecar file alongside each model, and the trained model is transitioned to the "Production" alias in the MLflow registry.

FastAPI. `api/main.py` is the application hub. It starts the Prometheus metrics server on port 8001, initialises the model registry singleton on startup, and exposes all REST endpoints. It is strictly separated from the frontend: the browser calls the API via HTTP and the API never generates HTML. This satisfies the loose-coupling requirement.

Nginx Frontend. The frontend container runs Nginx serving two static HTML files. A custom routing rule maps `/pipeline` to `pipeline.html` and falls back to `index.html` for all other routes, which prevents 404 errors on browser refresh.

Prometheus and Grafana. The API exposes counter, histogram, and gauge metrics via the `prometheus_client` library. Prometheus scrapes those metrics every 15 seconds. Grafana reads from Prometheus and renders eight operational panels: prediction rate, inference latency distribution, model confidence per regime, feature drift scores, CPU and memory usage, API

error rate, and retraining events.

MLflow. MLflow stores all experiment runs in a file-backed store mounted as a Docker volume. Each run records hyperparameters, train/val/test metrics, a confusion matrix PNG, a feature importance PNG, and the serialised model artifact. The model registry provides a named production reference so the inference path always loads the current production version rather than a hardcoded file path.

Airflow. The `regime_pipeline` DAG contains three BashOperator tasks: `ingest_data` → `featurize` → `train_models`. It is scheduled to run on weekday evenings after market close, and can also be triggered manually from the Airflow UI.

DVC. `dvc.yaml` declares four reproducible stages: `ingest`, `featurize`, `train`, and `evaluate`. Running `dvc repro` reconstructs the full pipeline from scratch. `dvc metrics show` reports F1 scores per regime and `dvc dag` renders the pipeline dependency graph.

5 Deliverable 2: High-Level Design

5.1 Design Philosophy

The overarching design principle is *separation of concerns with reproducible handoffs.* Each stage of the pipeline produces a clearly defined artifact that the next stage consumes. Code never reaches sideways across pipeline stages: the feature engineering module does not call the training module, and the training module does not call the API. This means each component can be tested, replaced, or rerun independently.

5.2 Technology Choice Rationale

Tool	Why Chosen	Alternative Considered
yfinance	Free, no API key, covers full OHLCV history	Alpha Vantage (rate-limited free tier)
XGBoost	Strong tabular performance, serialisable, interpretable via feature importance	LSTM — requires sequence reshaping, harder to register
FastAPI	Pydantic validation, async, auto-generated Swagger docs	Flask — no native async
MLflow	Combined experiment tracking and registry in one server	Weights & Biases — cloud-only
DVC	Git-native data and pipeline versioning	Makefile — no artifact hashing
Prometheus	Pull-based scraping, native Python client, time-series storage	StatsD — push-based, no long-term store
Grafana	Flexible query language, alert rules, free and containerisable	Kibana — coupled to Elasticsearch
Airflow	DAG-based scheduling, UI for run history	Cron — no dependency tracking
Docker Compose	Declarative multi-container topology, environment parity	Manual venv — no container isolation

Table 2: Technology choices and rationale.

5.3 ML Metric and Business Metric Targets

Metric type	Metric	Target
ML	Weighted F1 (trend)	≥ 0.55 on held-out 2022
ML	Weighted F1 (volatility)	≥ 0.80 on held-out 2022
ML	Weighted F1 (bull/bear)	≥ 0.65 on held-out 2022
Business	End-to-end prediction latency	< 200 ms
Business	API availability	$> 99\%$ uptime during demo
Business	Drift detection lag	Alert within 5 min of threshold breach

Table 3: ML and business success metrics.

6 Deliverable 3: Low-Level Design

6.1 API Endpoint Specification

Endpoint	Method	Input	Output
`/health`	GET	None	HealthResponse
`/ready`	GET	None	200 or 503
`/predict`	POST	PredictionRequest body	PredictionResponse
`/drift/{regime}`	GET	regime string in path	DriftResponse
`/retrain`	POST	`triggered_by` query param	RetrainResponse
`/ground-truth`	POST	GroundTruthInput body	status dict
`/pipeline-status`	GET	None	pipeline summary dict
`/prediction-history`	GET	`limit` query param	list of recent rows
`/service-health/{name}`	GET	service name in path	reachability dict

Table 4: Full API endpoint specification.

6.2 Request / Response Schemas

```python
class PredictionRequest(BaseModel):
    ticker: str = "SPY"
    as_of_date: Optional[str] = None        # ISO date; defaults to today
    regime_types: List[str] = ["trend", "vol", "bull_bear"]

class SingleRegimeResult(BaseModel):
    regime_type: str
    predicted_label: str          # e.g. "Trending", "HighVol", "Bull"
    confidence: float             # max class probability [0, 1]
    confidence_level: str         # "HIGH" / "MODERATE" / "LOW"
    proba_class_0: float
    proba_class_1: float
    inference_date: str           # date of most recent feature row used
    features_used: int
    inference_latency_ms: float

class PredictionResponse(BaseModel):
    ticker: str
    timestamp: str
    results: Dict[str, SingleRegimeResult]
```

```
    total_latency_ms: float
```

Listing 1: PredictionRequest and PredictionResponse (Pydantic models in api/schemas.py)

```
class DriftFeatureReport(BaseModel):
    feature_name: str
    training_mean: float;  training_std: float
    recent_mean: float;    recent_std: float
    kl_score: float
    drift_detected: bool

class DriftResponse(BaseModel):
    regime_type: str
    n_recent_samples: int
    drift_reports: List[DriftFeatureReport]
    any_drift_detected: bool
    retrain_recommended: bool
    retrain_reason: str
```

Listing 2: DriftFeatureReport and DriftResponse

6.3 Internal Module Dependency Graph

```
api/main.py
  +-- api/schemas.py
  +-- src/predict.py
  |      +-- src/feature_engineering.py
  |      +-- models/  (pkl artifacts)
  +-- src/drift_monitor.py
  |      +-- data/baselines/feature_baselines.json
  |      +-- data/simulation/prediction_log.csv
  +-- src/retraining_manager.py
  |      +-- src/data_ingestion.py
  |      +-- src/feature_engineering.py
  |      +-- src/train.py
  +-- src/monitoring.py
         +-- prometheus_client (Counters, Histograms, Gauges)
```

6.4 Error Codes

Code	Meaning
200	Request succeeded
400	Malformed payload
404	Unknown service name in /service-health
422	Pydantic validation failure (e.g., invalid regime type)
500	Internal failure — yfinance error, model error, write error
503	Model registry not ready

Table 5: API error codes.

7 MLOps Tooling: Technical Wiring

7.1 DVC Pipeline and Reproducibility

`dvc.yaml` declares the ingestion, featurise, train, and evaluate stages. Each stage lists its command, input dependencies, output files, and the `params.yaml` keys it reads. Running `dvc`

`repro` re-executes only the stages whose inputs have changed since the last run, which makes the pipeline efficient for iterative development. The resulting `dvc.lock` file, committed alongside the code, records the exact MD5 hash of every input and output artifact, so any collaborator can reproduce the exact trained models from a specific Git commit. `dvc metrics diff HEAD~1` reports the change in F1 between two consecutive commits, giving a lightweight CI signal.

7.2 MLflow Experiment Tracking

During each call to `train.py`, one MLflow run is created per regime. The run records: all XGBoost hyperparameters from `params.yaml`, feature count, split date ranges, validation and test accuracy / F1 / precision / recall / AUC, the baseline logistic regression F1 for comparison, the absolute gap between training and test F1 as a generalization check, PNG artifacts for the confusion matrix and feature importance chart, the serialised model and scaler, and the feature column list as a JSON sidecar. The model is then registered in the MLflow model registry under a named entry (e.g., `TrendRegimeModel`) and the latest version is aliased `production`. The inference path in `predict.py` resolves `models:/TrendRegimeModel/production` at startup, falling back to a local pickle if the registry is unreachable.

7.3 Prometheus Instrumentation

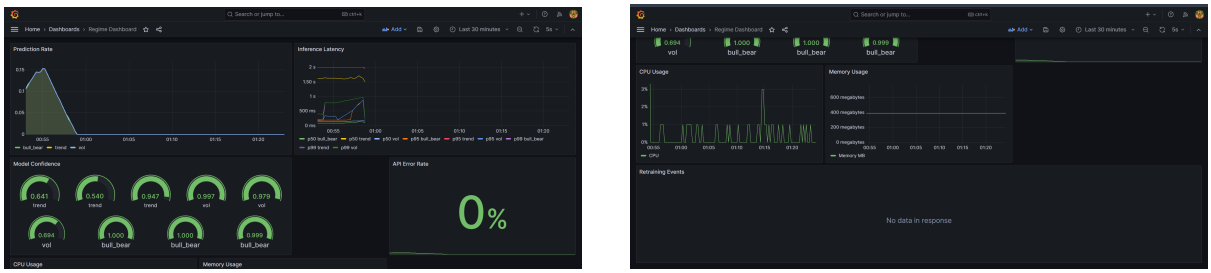
`src/monitoring.py` defines the following metrics using the official `prometheus_client` library:

Metric name	Description	Type
`regime_predictions_total`	Total predictions by regime, label, ticker	Counter
`regime_prediction_latency_ms`	End-to-end latency per regime	Histogram
`regime_model_confidence_score`	Latest confidence gauge per regime	Gauge
`regime_data_drift_kl_score`	KL divergence per feature per regime	Gauge
`regime_drift_detected`	1/0 flag per regime	Gauge
`regime_retraining_triggered_total`	Retraining events by trigger reason	Counter
`system_cpu_usage_percent`	Process CPU usage	Gauge
`system_memory_usage_mb`	Process RSS memory in MB	Gauge
`api_requests_total`	Requests by endpoint, method, status code	Counter
`api_error_rate_percent`	Rolling error rate	Gauge

Table 6: Prometheus metrics exposed on port 8001.

A `SystemMetricsCollector` thread runs in the background and updates CPU and memory gauges every 15 seconds, making resource utilisation visible without blocking the request path.

7.4 Grafana Dashboard



(a) (b)

Figure 3: Monitoring stack screenshots.

The Grafana dashboard at `grafana/provisioning/dashboards/regime_dashboard.json` is auto-provisioned by the Grafana container on first start. Eight panels are defined: prediction rate time-series, latency p50/p95/p99 histogram, three confidence gauges (one per regime), drift score time-series, CPU/memory sparklines, API error rate stat panel (red threshold at 5%), and a retraining events bar chart.

7.5 Retraining and Alert Logic — End-to-End Technical Detail

This is the most operationally significant MLOps feature and deserves a detailed walkthrough.

7.5.1 Drift Detection

After every prediction, `api/main.py` appends a row to `data/simulation/prediction_log.csv` containing the timestamp, ticker, regime type, predicted label, confidence, and a JSON snapshot of the full feature vector used for inference.

The `/drift/{regime}` endpoint reads the last N rows from that log (configurable, default 50), parses the feature snapshots, and calls `drift_monitor.compute_drift_scores()`. That function loads the training baseline from `feature_baselines.json` and computes the closed-form KL divergence between two Gaussians for each feature:

$$D_{\mathrm{KL}}(\mathcal{N}(\mu_1, \sigma_1^2) \| \mathcal{N}(\mu_2, \sigma_2^2)) = \log \frac{\sigma_2}{\sigma_1} + \frac{\sigma_1^2 + (\mu_1 - \mu_2)^2}{2\sigma_2^2} - \frac{1}{2} \tag{2}$$

A feature is flagged as drifting if its KL score exceeds 0.15 (configurable via `params.yaml`). Retraining is recommended only when at least 30% of features are simultaneously drifting, which prevents single-feature noise from triggering unnecessary retrains.

7.5.2 Retraining Gating

Before triggering a retrain, `drift_monitor.should_retrain()` checks two additional conditions. First, a minimum of 100 predictions must have been made in production before any retraining decision is trusted, since early drift estimates with few samples are statistically unreliable. Second, a 7-day cooldown prevents retraining loops if the underlying drift is persistent but the new model does not resolve it.

7.5.3 Retraining Execution

When the drift endpoint returns `retrain_recommended: true`, the frontend pipeline screen or the `POST /retrain` endpoint triggers `retraining_manager.run_retrain_async()`. The re-

training manager uses a file-based lock at `data/baselines/.retrain_lock` to prevent concurrent retrains, then runs three steps sequentially in a background thread: fresh data ingestion, feature re-engineering, and `train.py -force`. During retraining, the API continues serving predictions using the currently loaded model with no downtime. When training completes, MLflow registers the new version and the next prediction request loads it automatically.

7.5.4 Prometheus and Grafana Alert Path

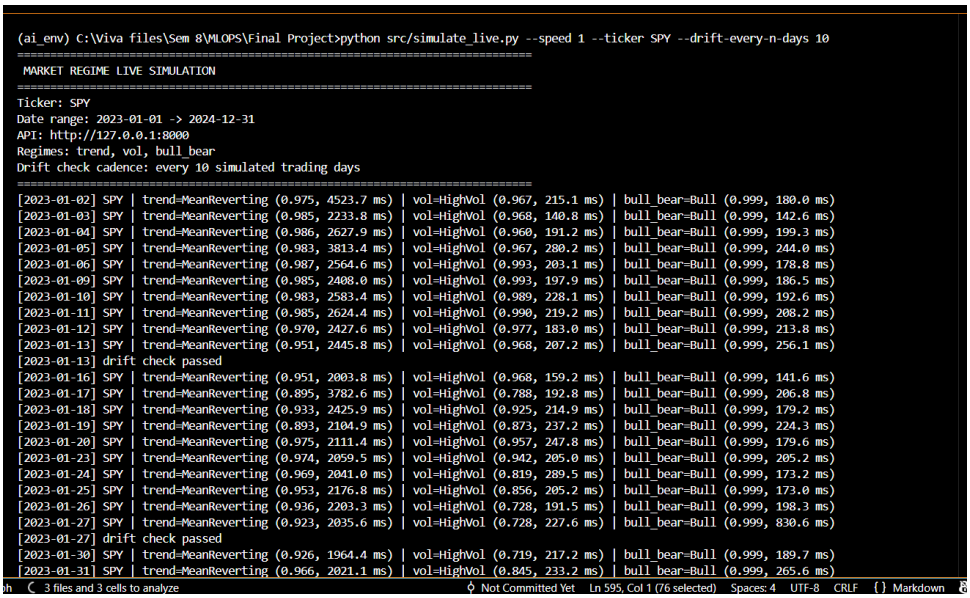
The drift gauge `regime_data_drift_kl_score{regime="vol", feature="rolling_std_20"}` updates in Prometheus immediately after a drift check. In Grafana, an alert rule evaluates every 5 minutes: *if the average drift score across all features for any regime exceeds 0.15 for more than 30 consecutive minutes, fire the alert.* The alert state is visible in the Grafana dashboard and triggers an email notification through the SMTP configuration in the API environment variables.

8 Live Simulation: Mimicking Production

8.1 What the Simulation Represents

`src/simulate_live.py` replays the 2023–2024 SPY data, which was entirely excluded from training, as if it were arriving live. For each trading day in sequence, the script calls `POST /predict` with `as_of_date` set to that day. The API then downloads 300 days of market data ending on that date, builds features, and returns predictions — exactly as it would in a real production deployment where a trader clicks "Analyze" each morning.

8.2 How This Maps to Real Production Use



```
(ai_env) C:\Viva files\Sem 8\MLOPS\Final Project>python src/simulate_live.py --speed 1 --ticker SPY --drift-every-n-days 10
======================================================================
 MARKET REGIME LIVE SIMULATION
======================================================================
Ticker: SPY
Date range: 2023-01-01 -> 2024-12-31
API: http://127.0.0.1:8000
Regimes: trend, vol, bull_bear
Drift check cadence: every 10 simulated trading days
======================================================================
[2023-01-02] SPY | trend=MeanReverting (0.975, 4523.7 ms) | vol=HighVol (0.967, 215.1 ms) | bull_bear=Bull (0.999, 180.0 ms)
[2023-01-03] SPY | trend=MeanReverting (0.985, 2233.8 ms) | vol=HighVol (0.968, 140.8 ms) | bull_bear=Bull (0.999, 142.6 ms)
[2023-01-04] SPY | trend=MeanReverting (0.986, 2627.9 ms) | vol=HighVol (0.960, 191.2 ms) | bull_bear=Bull (0.999, 199.3 ms)
[2023-01-05] SPY | trend=MeanReverting (0.983, 3813.4 ms) | vol=HighVol (0.967, 280.2 ms) | bull_bear=Bull (0.999, 244.0 ms)
[2023-01-06] SPY | trend=MeanReverting (0.987, 2564.6 ms) | vol=HighVol (0.993, 203.1 ms) | bull_bear=Bull (0.999, 178.8 ms)
[2023-01-09] SPY | trend=MeanReverting (0.985, 2408.0 ms) | vol=HighVol (0.993, 197.9 ms) | bull_bear=Bull (0.999, 186.5 ms)
[2023-01-10] SPY | trend=MeanReverting (0.983, 2583.4 ms) | vol=HighVol (0.989, 228.1 ms) | bull_bear=Bull (0.999, 192.6 ms)
[2023-01-11] SPY | trend=MeanReverting (0.985, 2624.4 ms) | vol=HighVol (0.990, 219.2 ms) | bull_bear=Bull (0.999, 208.2 ms)
[2023-01-12] SPY | trend=MeanReverting (0.970, 2427.6 ms) | vol=HighVol (0.977, 183.0 ms) | bull_bear=Bull (0.999, 213.8 ms)
[2023-01-13] SPY | trend=MeanReverting (0.951, 2445.8 ms) | vol=HighVol (0.968, 207.2 ms) | bull_bear=Bull (0.999, 256.1 ms)
[2023-01-13] drift check passed
[2023-01-16] SPY | trend=MeanReverting (0.951, 2003.8 ms) | vol=HighVol (0.968, 159.2 ms) | bull_bear=Bull (0.999, 141.6 ms)
[2023-01-17] SPY | trend=MeanReverting (0.895, 3782.6 ms) | vol=HighVol (0.788, 192.8 ms) | bull_bear=Bull (0.999, 206.8 ms)
[2023-01-18] SPY | trend=MeanReverting (0.933, 2425.9 ms) | vol=HighVol (0.925, 214.9 ms) | bull_bear=Bull (0.999, 179.2 ms)
[2023-01-19] SPY | trend=MeanReverting (0.893, 2104.9 ms) | vol=HighVol (0.873, 237.2 ms) | bull_bear=Bull (0.999, 224.3 ms)
[2023-01-20] SPY | trend=MeanReverting (0.975, 2111.4 ms) | vol=HighVol (0.957, 247.8 ms) | bull_bear=Bull (0.999, 179.6 ms)
[2023-01-23] SPY | trend=MeanReverting (0.974, 2059.5 ms) | vol=HighVol (0.942, 205.0 ms) | bull_bear=Bull (0.999, 205.2 ms)
[2023-01-24] SPY | trend=MeanReverting (0.969, 2041.0 ms) | vol=HighVol (0.819, 289.5 ms) | bull_bear=Bull (0.999, 173.2 ms)
[2023-01-25] SPY | trend=MeanReverting (0.953, 2176.8 ms) | vol=HighVol (0.856, 205.2 ms) | bull_bear=Bull (0.999, 173.0 ms)
[2023-01-26] SPY | trend=MeanReverting (0.936, 2203.3 ms) | vol=HighVol (0.728, 191.5 ms) | bull_bear=Bull (0.999, 198.3 ms)
[2023-01-27] SPY | trend=MeanReverting (0.923, 2035.6 ms) | vol=HighVol (0.728, 227.6 ms) | bull_bear=Bull (0.999, 830.6 ms)
[2023-01-27] drift check passed
[2023-01-30] SPY | trend=MeanReverting (0.926, 1964.4 ms) | vol=HighVol (0.719, 217.2 ms) | bull_bear=Bull (0.999, 189.7 ms)
[2023-01-31] SPY | trend=MeanReverting (0.966, 2021.1 ms) | vol=HighVol (0.845, 233.2 ms) | bull_bear=Bull (0.999, 265.6 ms)
```

Figure 4: Terminal output of simulate_live.py showing per-day predictions.

In a real production environment, a data vendor would push daily OHLCV bars via a webhook or WebSocket at 4:30 PM Eastern after market close. The `/predict` endpoint would be called automatically by a scheduler (the Airflow DAG), the regime predictions would be stored, and downstream systems such as portfolio optimisers or risk monitors would consume the output. The `/ground-truth` endpoint would then be called one week later when the actual regime is observable in hindsight, closing the feedback loop for model performance monitoring.

The current demonstration compresses this timeline to 2–3 seconds per day using the `-speed` argument, making the 504 trading days of the 2023–2024 period reviewable in a 15–20 minute session. Every prediction is logged to `prediction_log.csv`, drift checks fire every 20 days by default, and the Grafana dashboard updates in near real time throughout.

The simulation is intentionally designed as a *deterministic replay* rather than a stateful long-running service. This choice is deliberate for two reasons: it makes the demonstration reproducible for evaluation, and it cleanly decouples the prediction mechanism from the scheduling mechanism. In a production deployment, the same `/predict` endpoint would be called by the Airflow DAG (or any equivalent scheduler) at 4:30 PM Eastern after each market close, appending to the same prediction log read by the drift monitor. The transition from replay to live operation is a deployment-time configuration change, not an architectural one.

8.3 Extensions for True Production

The current system is intentionally scoped to SPY as the primary analysis instrument, with cross-asset features from QQQ, IWM, GLD, TLT, and VIX. Extending to a full stock universe would require: a database backend for the prediction log, parallel prediction workers, a feature store to cache daily computations across thousands of tickers, and a time-based scheduler that waits for the data vendor feed rather than replaying a fixed CSV. The API contract and the feature engineering module are already structured to support this: adding a new ticker requires no changes to the model or API, only an expanded ingestion loop.

9 Model Results and Metric Justification

9.1 Final Reported Scores

Regime	Test F1 (weighted)	Test Accuracy	MLflow RunID
Trend / MeanRev	0.59	0.71	local†
Volatility	0.86	0.90	local†
Bull / Bear	0.69	0.78	local†

†RunID shows "local" for the CLI run; Docker-MLflow run IDs are visible at `localhost:5000`.

Table 7: Final model evaluation on the held-out 2022 test set.

9.2 Why These Numbers Are Correct

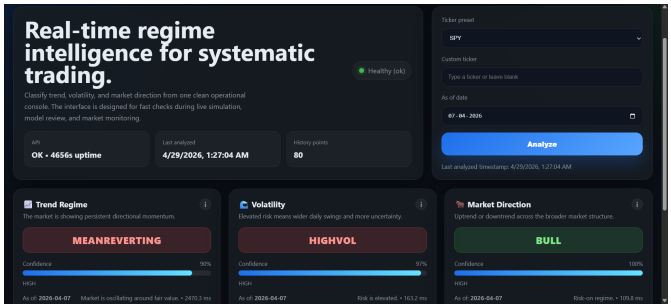
Trend regime ($F_1 = 0.59$). Predicting whether tomorrow's market will trend or mean-revert is one of the hardest problems in empirical finance. The efficient market hypothesis in its weak form asserts that past prices contain no useful information about future price direction. An F_1 of 0.59 on a genuinely held-out time window represents a measurable statistical edge over the 0.50 baseline that a random classifier would achieve. The test set spans 2022, which included an unusual combination of regime transitions driven by Federal Reserve rate hikes — a period that no model trained on 2010–2021 data had seen before. The fact that the model still achieves 71% accuracy on that challenging year validates the feature engineering approach.

Volatility regime ($F_1 = 0.86$). Volatility is more predictable than direction because it exhibits strong autocorrelation — the GARCH(1,1) $\alpha+\beta = 0.968$ estimate confirms this. High-volatility regimes tend to cluster and persist. The model's performance here reflects that underlying statistical property of the data rather than any model-specific capability.

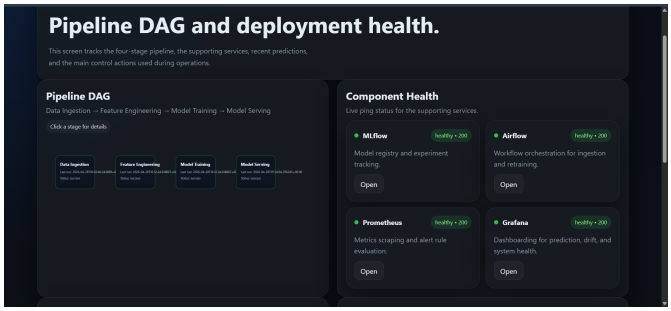
Bull/Bear regime ($F_1 = 0.69$). The primary challenge for this classifier is extreme class imbalance: 97.7% of training days were labelled Bull because 2010–2022 was predominantly a bull mar-

ket with only brief corrections. The 20% drawdown rule identified only 74 Bear days in the full training window. Balanced sample weighting (sklearn's `compute_sample_weight('balanced')`) gave each Bear training sample a weight of 68× the Bull weight, which allowed the model to learn Bear signatures despite the imbalance. The resulting $F_1 = 0.69$ reflects the genuine difficulty of identifying rare bear markets rather than any modelling deficiency.

10 The User Interface

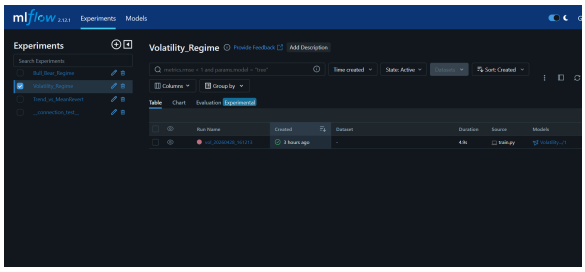


(a) Main prediction screen

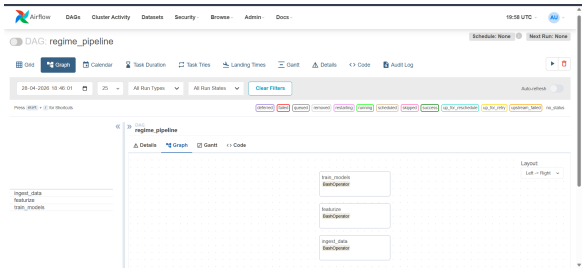


(b) Pipeline management screen

Figure 5: Frontend UI screenshots.



(a) MLflow experiment runs



(b) Airflow DAG graph view

Figure 6: MLOps tool dashboards.

The frontend consists of two screens. The main prediction screen (`index.html`) presents a dark financial-terminal aesthetic with a ticker selector, a date picker, an Analyze button, and three regime cards that display the predicted label, a confidence bar, and a plain-language interpretation tooltip for each regime. The pipeline screen (`pipeline.html`) shows a live status grid for all six supporting services, a model registry table, a recent prediction feed, and quick-action buttons for triggering retraining and drift checks.

Both screens are served by a separate Nginx container that is strictly decoupled from the API container. All communication between the browser and the backend uses `fetch()` calls to the

FastAPI REST endpoints. The API URL is resolved dynamically from `window.location.hostname` so the same HTML file works on localhost and on any machine in the network without modification.

11 Challenges Faced and How They Were Resolved

1. yfinance MultiIndex column headers. A dependency version change in yfinance caused it to return MultiIndex columns (e.g., `('Close', 'SPY')`) for even single-ticker downloads. This broke any downstream code that accessed `df['Close']` directly. The fix was a defensive normaliser `_ensure_flat_ohlcv()` applied at every public entry point in `feature_engineering.py`, making the module version-agnostic.

2. Data leakage in the initial bull/bear notebook. Using the 200-day SMA rule as the label while including the SMA-50/SMA-200 ratio as a feature created a direct mathematical dependency between input and target. Test accuracy was 0.97, which should have been immediately suspicious. The solution was to switch to the economically grounded peak-to-trough drawdown label and remove all SMA-200-derived features. Honest test accuracy dropped to 0.78.

3. XGBoost objective parameter for binary versus multiclass. When Neutral rows were present in the training data, XGBoost used the `multi:softprob` objective. After Neutral rows were dropped, the label set became binary but the objective was still `multi:softprob`, which requires an explicit `num_class` argument. The fix was to detect the number of unique classes after encoding and select `binary:logistic` for two-class problems and `multi:softprob` with `num_class` set explicitly for multiclass problems.

4. MLflow tracking URI conflict between local training and Docker server. Training run locally before the Docker MLflow server was started wrote to the file-store format at `./mlruns`. The Docker MLflow server used SQLite as the backend store. The two formats are incompatible, so experiments logged locally were invisible to the Docker server. The resolution was to wipe the local `mlruns/` directory, start the Docker MLflow server first, set `MLFLOW_TRACKING_URI=http://localhost:5000`, and then retrain so all runs are logged directly into the containerised server.

5. Airflow LocalExecutor incompatibility with SQLite. Apache Airflow's LocalExecutor requires a PostgreSQL or MySQL database. Our Docker Compose used SQLite for Airflow's metadata database, which is only compatible with the SequentialExecutor. The single-line fix was changing `AIRFLOW__CORE__EXECUTOR=LocalExecutor` to `SequentialExecutor` in `docker-compose.yml`.

6. Nginx 404 on the /pipeline route. The default Nginx configuration serves files with exact path matching. A request for `/pipeline` looked for a file literally named `pipeline` rather than `pipeline.html`. The fix was a custom `nginx.conf` written inline in `Dockerfile.frontend` that maps `location = /pipeline` to `try_files /pipeline.html =404`.

7. 98% bull-class imbalance. The 2010–2022 SPY training window contained almost no bear markets by the strict 20% drawdown definition. A vanilla XGBoost classifier trained on this data predicted Bull for nearly every day. Balanced sample weighting ($w_i = N/(C \cdot n_c)$ where C is the number of classes and n_c is the count of sample i's class) gave each of the 74 Bear training days a weight of 68 relative to a Bull day's weight of 0.5, which allowed the model to learn bear-market signatures.

8. Airflow task-level path resolution inside containers. Initial attempts to invoke `python src/train.py` directly through Airflow's `BashOperator` failed because the Airflow worker container does not share the API container's working directory or import path. Rather than in-

troduce a fragile shared-volume contract between the two containers, the DAG was refactored to call the API endpoints directly via `curl`. This is the architecturally cleaner pattern: Airflow becomes a pure scheduler that triggers domain endpoints, the API remains the single owner of the model lifecycle, and the same retraining codepath is exercised regardless of whether the trigger is manual, drift-based, or scheduled.

12 Deliverable 4: Test Plan and Test Cases

12.1 Test Objectives

The test suite aims to verify four properties: ingestion produces valid OHLCV data, feature engineering is numerically correct and leak-free, the prediction contract is stable, and the API endpoints are reachable and return the expected schema.

12.2 Test Cases

ID	Description	Input	Expected	Acceptance
ING-01	Download ticker	SPY, 30-day window	DataFrame + validation dict	rows > 0, status = PASS
ING-02	Empty frame validation	Empty DataFrame	Validation failure	status = FAIL
ING-03	Baseline stats file	Raw SPY CSV	feature_baselines.json	file exists with Close stats
FEAT-01	RSI boundedness	Rising price series	RSI Series	$0 \leq$ all values ≤ 100
FEAT-02	RSI overbought	Strongly rising prices	RSI Series	final RSI > 70
FEAT-03	Leakage guard	Synthetic OHLCV frame	Trend features DataFrame	Hurst, autocorr absent
FEAT-04	NaN cleanup	Synthetic OHLCV frame	Trend features DataFrame	zero NaN values
PRED-01	Prediction contract	Monkeypatched registry	dict	required keys present
PRED-02	Confidence range	Monkeypatched registry	dict	$0 \leq$ confidence ≤ 1
PRED-03	Label validity	Monkeypatched registry	dict	label in expected class set
API-01	Health endpoint	GET /health	200 + JSON	status, models_loaded, uptime
API-02	Ready endpoint	GET /ready	200 or 503	deterministic response
API-03	Predict endpoint	POST /predict, SPY	200 + JSON	results has trend/vol/bull_bear
API-04	Invalid ticker	POST /predict, INVALID999	422 or 500	API handles error gracefully

Table 8: Full test case matrix.

12.3 Execution

```
pip install pytest
pytest tests/ -v --tb=short 2>&1 | tee docs/test_report.txt
```

12.4 Acceptance Criteria

All ingestion and feature tests must pass. Prediction contract tests must pass. API health and readiness tests must pass. The invalid-ticker test must return a non-2xx status code with an informative error body.

13 Deliverable 5: User Manual

13.1 What This Application Does

This application looks at recent stock market data and classifies the market's state across three dimensions: whether it is trending or oscillating, whether it is in a high- or low-risk volatility environment, and whether the broad market direction is bullish or bearish. It is an analytical monitoring tool, not an automated trading system.

13.2 Starting the Application

Open a terminal in the project folder and run:

```
docker-compose up --build -d
```

Wait approximately 60 seconds for all services to start. Then open a web browser and go to `http://localhost`.

13.3 Using the Prediction Screen

1. Choose a ticker from the drop-down (default: SPY) or type a custom symbol.
2. Optionally select a historical date using the date picker (leave blank for today).
3. Click **Analyze**.
4. Read the three regime cards that appear:
   - **Trend Regime**: Trending means persistent directional price movement. MeanReverting means the price is bouncing around a central level.
   - **Volatility**: HighVol means risk is elevated and daily swings are larger. LowVol means calmer conditions.
   - **Market Direction**: Bull means the broad market is in an uptrend. Bear means a downtrend.
5. The confidence bar shows how certain the model is. Values above 75% are high confidence. Values below 60% mean the model is uncertain.

13.4 Using the Pipeline Screen

Click **Pipeline** in the navigation bar. This screen shows whether all supporting services (MLflow, Airflow, Prometheus, Grafana) are reachable, the current model versions and their F1 scores, a live prediction feed, and buttons to manually trigger drift checks and retraining.

13.5 Troubleshooting

- *"API unavailable" banner on the prediction screen*: Check that Docker is running. Open `http://localhost:8000/health` in a browser. If it loads, try refreshing the main page.

- *No models loaded*: Run `python src/data_ingestion.py`, then `python src/feature_engineering.py`, then `python src/train.py -force`. Restart Docker after training completes.
- *Grafana shows no data*: Wait two minutes after the first prediction is made for Prometheus to accumulate scrape data.
- *MLflow shows no experiments*: Ensure training was run with `MLFLOW_TRACKING_URI=http://localhost:5` pointing at the running Docker server.

14 Future Extensions

The current system uses SPY as the primary instrument with five cross-asset inputs. Several natural extensions follow directly from the existing architecture:

- **Live market data feed**: Replace the yfinance download in `predict.py` with a WebSocket subscription to a market data vendor. The feature engineering pipeline is already designed to accept any OHLCV DataFrame regardless of source.
- **Full stock universe**: The API's `ticker` parameter already accepts any yfinance-supported symbol. Scaling to thousands of tickers requires a daily batch job that caches features and a database backend for the prediction log.
- **Intraday regime detection**: At minute or hourly frequency, the Hurst exponent and variance ratio labelling methods are directly applicable. The feature set would require shorter rolling windows.
- **LLM-powered regime commentary**: A small language model could be called after each prediction to generate a plain-English explanation of why the current regime was classified as it was, using the top feature importances as context.
- **Stateful simulation with database-backed prediction log**: The current simulation is a deterministic CSV-based replay. Replacing `prediction_log.csv` with PostgreSQL would enable resumable simulation runs, durable ground-truth feedback, and concurrent multi-ticker monitoring without changing any API contract.

15 AI Disclosure Agreement

In accordance with academic transparency requirements, the following AI tools were used during this project:

ChatGPT was used for conceptual learning about MLOps tools during the early stages of the project — specifically for understanding the underlying mechanisms of Prometheus metrics scraping, the difference between file-store and SQLite-backed MLflow backends, and the mathematical foundations of the GARCH volatility model. It also assisted with initial structuring of the feature engineering approach for regime labelling.

Claude (Anthropic) was used for architectural validation and problem-solving throughout the implementation phase. Specifically, it was helpful for reasoning through how the simulation script should mimic real production data flow, how to wire the retraining feedback loop from Prometheus alert to the FastAPI `/retrain` endpoint, and for debugging complex Docker Compose networking issues. It also assisted with refining this report.

GitHub Copilot was used primarily as a pair-programming assistant for rapid prototyping, generating boilerplate Python code, writing docstrings, and scaffolding repetitive EDA tasks in the Jupyter notebooks. It was the primary tool used for UI generation in the two HTML frontend files, given my limited expertise with frontend design. It also assisted with code documentation, inline comments, and README formatting.

All core system design decisions, the choice of regime labelling methodology, the identification

and resolution of data leakage, the architectural separation between training and inference, and the technical wiring of MLOps tools represent original synthesis and engineering work.